

RAG 시스템 백엔드 핵심 로직 설명서

본 문서는 법률 문서에 특화된 Retrieval-Augmented Generation(RAG) 시스템의 백엔드 파이프라인 중 핵심 로직에 대한 설계 및 구현 기술 설명서입니다. 단위테스트 결과서를 기반으로 도출된 핵심 모듈의 동작 원리와 예외 처리 구조를 상세히 기술합니다.

1. 개요 (Overview)

문서파일 업로드 후 청킹및요약+대용량 법률 데이터의 전처리, 정제, 임베딩, 그리고 데이터베이스(벡터 DB 및 RDB) 간의 정합성 보장을 담당하는 백엔드 인프라의 핵심 기술을 서술합니다.

2. 핵심 로직 상세 분석 (Core Logic Analysis)

2.1. 업로드 파일 확장자별 텍스트추출 알고리즘 (UT-DPP)

업로드 파일마다 텍스트 추출 방식은 pdf로 변환후 텍스트추출하는 로직입니다 만 hwp,hwpX는 rhwp를 활용하여 ocr거치지않고 텍스트추출하게됩니다.

항목	상세 내용
관련 테스트 ID	UT-DPP
핵심 함수 및 클래스/변수	함수 detect_doc_type(), extract_text_from_file(),
구현 핵심 로직	파일 확장자마다 분기별로 나뉘게 됩니다. 텍스트 추출하는 부분은 확장자마다 다르나 hwp와 txt파일을 제외한 파일들은 pdf로 변환후 pdf문서에서 텍스트추출하게됩니다.

2.2. PDF변환후 텍스트 추출(UT-OCR)

PDF로 변환을 한후 텍스트추출하는 과정입니다.

항목	상세 내용
관련 테스트 ID	UT-OCR
핵심 함수 및 클래스/변수	함수 <code>process_pdf(),extract_native_blocks(),extract_image_regions(),remove_duplicate_ocr()</code> 변수 <code>FLAGS_use_mkldnn,FLAGS_use_onednn, FLAGS_enable_pir_api</code>
구현 핵심 로직	pdf로 변환된 문서파일을 이미지,텍스트 구분해서 나눠서 처리합니다. 이미지에서 나온 텍스트가 기존에 추출된 텍스트와 중복이있다면 중복을 없애고 텍스트와 이미지에서 추출된 텍스트와 합칩니다. 그리고 페이지별 텍스트를 만들어서 결과값을 저장합니다.

2.3. 법률 특화 구조화 및 문맥 상속 청킹 흐름(UT-DPL, UT-CHK)

불필요한 기호와 괄호를 정제한뒤 법률문서의 고유구조인 조단위로 청킹을 하는 로직입니다.

항목	상세 내용
관련 테스트 ID	UT-DPL계열 및 UT-CHK계열
핵심 함수 및 클래스/변수	클래스 Chunk, ChunkConfig 함수 <code>extract_article()</code> 변수 <code>_RE_LEGAL_SECTIONS, _RE_ARTICLE_H</code> <code>EADER, _RE_PURE_ARTICLE</code> <code>use_law_structure:bool</code>
구현 핵심 로직	본문의 조(Article) 단위를 식별하여 청킹을 수행합니다. 정규식을 통해 본문 내에서 조 경계를 찾아 분할하며, 분할된 청크에는 '법령명 + 제1조' 형식으로 상위 문맥을 결합하여 추출합니다. 또한 <code>use_law_structure</code> 변수값에 따라 법률 문서 구조일 때와 일반 문서일 때의 청킹 알고리즘을 분기하여 유연하게 대응합니다.

2.4. 임베딩 및 인덱스 배치 흐름(UT-EMB, UT-IDX)

고성능 RAG 구현을 위해 Dense(밀집) 및 Sparse(희소) 임베딩을 동시에 생성합니다. Sparse 가중치 연산 시 노이즈를 제어하기 위해 특정 임계값(`SPARSE_THRESHOLD`) 이하의 값은 필터링하고 이를 다시 정규화(Normalization)합니다. 이후 대용량 벡터 업서트 시 부하를 막기 위해 데이터를 적정 크기의 배치로 분할(`_upsert_in_batches`)합니다.

항목	상세 내용
관련 테스트 ID	UT-EMB, UT-IDX
핵심 함수 및 클래스/변수	함수 <code>_parse_sparse()</code>, <code>_upsert_in_batches()</code>, <code>embed_texts()</code>, <code>build_pinecone_vectors()</code>, <code>sync_public_if_newer()</code> 변수 <code>SPARSE_THRESHOLD</code>
구현 핵심 로직	Dense와 Sparse 두 가지 형태의 하이브리드 벡터값으로 임베딩을 수행합니다. Sparse 가중치 연산 시 노이즈를 제거하기 위해 <code>SPARSE_THRESHOLD</code> 이하의 값은 필터링한 후 정규화(Normalization) 처리를 거치며, 최종적으로 대용량 벡터 업서트 시 발생할 수 있는 인프라 부하를 방지하기 위해 데이터를 적정 크기의 배치 단위(<code>_upsert_in_batches</code>)로 분할하여 처리합니다.

2.5. 인덱스 업서트 및 유저나 공용 네임스페이스 흐름(UT-CLI, UT-UPS)

외부 벡터 DB(Pinecone) 커백션의 오버헤드를 막기 위해 싱글톤(Singleton) 패턴으로 인스턴스 생성을 1회로 제한하고, 네임스페이스 격리 및 하이브리드 검색 가중치(alpha 스케일링)를 제어하는것.

항목	상세 내용
관련 테스트 ID	UT-CLI, UT-UPS
핵심 함수 및 클래스/변수	함수 <code>get_index()</code>, <code>delete_by_document_id</code>, <code>delete_all()</code>, <code>delete_by_ids()</code>, <code>upsert_dataset()</code> 메서드 <code>db.rollback()</code> 변수 <code>alpha</code>
구현 핵심 로직	외부 벡터 DB(Pinecone) 커백션 오버헤드를 막기 위해 싱글톤(Singleton) 패턴으로 인스턴스 생성을 1회로 제한하고, <code>public/user_{user_id}</code> 네임스페이스 격리로 공개·사용자 벡터를 분리합니다. 하이브리드 검색 시 <code>dense × alpha, sparse × (1-alpha)</code> 스케일링으로 검색 가중치를 제어하며, 빈 ID 리스트 전달 시 Pinecone API 호출 없이 조기 리턴합니다. 대량 청크 업서트 시 Pinecone upsert가 실패하면 RDB(PostgreSQL)에 삽입된 레코드를 즉시 롤백해 양측 데이터 정합성을 보장합니다.

2.6.데이터셋의 흐름(UT-CCR + UT-CDS)

벡터DB에 미리 적재하는 데이터셋의 흐름으로 `chunk_dataset`에 청크텍스트들을 적재하며, `vector_id`값으로 조회해 삭제하는 로직입니다.

항목	상세 내용
관련 테스트 ID	UT-CCR + UT-CDS
핵심 함수 및 클래스/변수	함수 <code>get_chunks_by_document()</code>, <code>create_chunks_bulk()</code>, <code>delete_chunks_from_rdb()</code>, <code>create_chunk_dataset()</code>, <code>get_dataset_chunk_by_vector_id()</code>, <code>update_chunk_dataset_text()</code>, <code>delete_dataset_chunk_by_vector_id()</code> 변수 <code>_parse_law_date()</code>
구현 핵심 로직	관계형 DB(PostgreSQL)에 청크 데이터를 저장할 때 사용자 업로드 청크(<code>chunks</code> 테이블, UT-CCR)와 사전 로드 데이터셋 청크(<code>chunk_dataset</code> 테이블, UT-CDS)를 분리하여 관리합니다. 두 테이블 모두 삽입 전 입력 길이 불일치 시 DB 호출 없이 즉시 <code>ValueError</code> 를 발생시키며, <code>db.commit()</code> 실패 시 <code>db.rollback()</code> 을 호출해 부분 삽입을 방지합니다. 날짜 필드(<code>law_date</code>)는

항목	상세 내용
	YYYY-MM-DD 포맷만 허용하고 잘못된 포맷은 None으로 폴백하며, UUID 기반 vector_id로 Pinecone과의 참조 정합성을 유지합니다.

2.7.요약청킹 텍스트 합본(UT-SUM)

추출된 텍스트를 요약 텍스트를 만들기위함으로 3개로 분할하여 진행합니다. 앞단,중간,마지막 최종카테고리또한 이과정에서 나오게됩니다.

항목	상세 내용
관련 테스트 ID	UT-SUM
핵심 함수 및 클래스/변수	함수 <pre>summarize(), summarize_document(), _sample_chunks(), _build_sample_text(), _extract_revised_date_regex(), _parse_response()</pre> 변수 <pre>MAX_SAMPLE_CHUNKS, MAX_ANALYSIS_CHARS, LAW_CATEGORIES, LAW_UNSTRUCTURED</pre>
구현 핵심 로직	추출된 문서 텍스트를 LLM에 전달하기 전, 토큰 비용 절감을 위해 전체 청크 중 앞 40% / 중간 20% / 뒤 40% 비율로 대표 청크를 샘플링하고 MAX_ANALYSIS_CHARS 이내로 절단해 분석용 텍스트를 조립합니다. 로컬 LLM(Ollama)을 통해 문서 카테고리 와 요약 을 추출하며, 카테고리가

항목	상세 내용
	LAW_CATEGORIES(법령·규정 / 판결문·결정문 / 계약서·협약서/행정문서·공문)에 해당할 경우에만 청킹·임베딩을 진행합니다. 개정일은 LLM에 의존하지 않고 정규식으로 키워드(개정, 시행 등) 주변을 우선 탐색해 추출하며, LLM 응답 실패 시에도 정규식으로 추출한 날짜는 보존합니다.

3. 성과 및 결론 (Achievements & Conclusion)

- 안정성 향상: 네트워크 장애 및 대량 데이터 삽입 실패 조건 검증을 통한 예외 처리율 100% 달성.
- 품질 최적화: 법률 특화 계층 파싱을 통해 RAG 컨텍스트 반환의 정확도 및 모델 답변 노이즈 대폭 완화.
- 비용 효율화: 외부 API 커넥션 싱글턴화 및 불필요 Sparse 임베딩 필터링으로 인프라 오버헤드 최적화.
- 문서 처리 범용성 확보: PDF·PPTX·DOCX·HWP·TXT 등 다양한 포맷의 텍스트 추출 경로를 단위 테스트로 검증하였고, Native 텍스트와 OCR 결과의 중복 제거 로직을 확인함으로써 포맷 무관 안정적 텍스트 추출 파이프라인을 완성.
- AI 분석 신뢰성 강화: 많은 양의 토큰 사용을 줄이기 위해 컨텍스트 샘플링 기반을 통해서 토큰을 절감하였고, 정규식 기반 개정일 독립적으로 추출, LLM 응답 실패시 날짜값은 보존하게되며, LLM의존도를 낮추는 방어 로직을 단위테스트로 검증해서 요약 파이프라인의 신뢰성을 확보하였습니다.